

Building a Semantic Search Engine for legal documents with Qdrant + LangExtract

10 min read · Aug 16, 2025



Vlds

Follow



Listen



Share

... More

In the legal world, precision is everything. Whether you're a lawyer searching for precedent or a researcher exploring case law, finding the exact clause or order in hundreds of documents can feel like searching for a needle in a haystack.

Traditional keyword search often falls short — it can miss relevant results hidden behind different phrasing, or return too many irrelevant hits.

In this project, I set out to solve that problem by combining Google's recently released **LangExtract** — a powerful open-source library for structured information extraction from long, unstructured text — with **Qdrant**, a high-performance vector database for semantic search, and a reranking layer for extra precision. The result is an intelligent, context-aware legal search pipeline that not only finds the right information but also shows it in its full context.

Why LangExtract?

When working with legal documents, one of the biggest challenges is extracting structured, reliable information from thousands of pages of dense, unstructured text. Traditional methods — regex, keyword search, or even naive LLM prompts — often struggle with accuracy, consistency, and traceability. LangExtract, a newly released open-source Python library from Google, addresses these challenges head-on.

LangExtract is built for schema-guided, large language model-driven extraction. It lets you define exactly what entities you want — such as *Parties*, *Orders*, *Citations*,

Procedural Histories, and *Case Names* — and ensures that every extracted item is tied back to its exact location in the original text through precise source grounding. This means you don't just get structured outputs — you also get the character offsets, so you can trace any fact directly back to the sentence it came from.

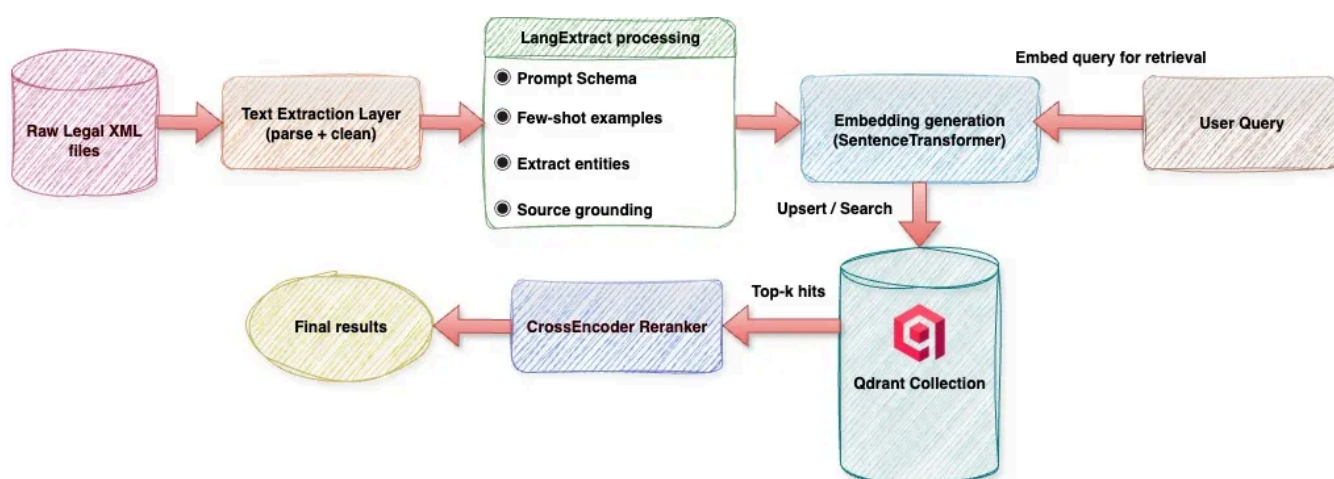
It also excels in long-context scenarios like court judgments, using chunking and parallel extraction to process massive documents without losing context. In our project, this allowed us to run extraction across hundreds of legal XML files, automatically build a semantic search index in Qdrant, and later retrieve contextually relevant legal facts — while preserving their original evidence text for verification.

LangExtract's flexibility means it works equally well with Google's Gemini models and open-source LLMs, making it a powerful bridge between raw unstructured legal text and structured, queryable legal intelligence.

Architecture

The legal information retrieval pipeline combines LangExtract for entity extraction, Qdrant for semantic indexing, and a re-ranker for improving retrieval quality. The architecture ensures that extracted legal facts remain traceable, searchable, and ranked by relevance.

Below is the flow of data from raw legal XML files to ranked search results:



The pipeline for semantic legal information retrieval works as follows:

- **Raw legal XML files** — The process starts with a collection of XML legal documents, containing both structured tags and unstructured text.

- **Text extraction layer (parse + clean)** — Parses the XML, removes noise and boilerplate, and isolates the main body of text while preserving character offsets for later grounding.
- **LangExtract processing** — Uses Google's LangExtract library to define prompts and output schemas for target entities, supply few-shot examples for consistent extraction, and extract entities while linking them back to their exact character offsets in the original text for full traceability.
- **Embedding generation** — Converts extracted entities into vector embeddings using SentenceTransformers, and embeds user queries into the same vector space for semantic retrieval.
- **Qdrant Vector Store** — Stores entity embeddings in Qdrant, supports upserts, and retrieves Top-k hits for each embedded query.
- **CrossEncoder Reranker** — Reranks the Top-k hits from Qdrant by evaluating the query and candidate text together for fine-grained semantic relevance.
- **Final results** — Returns the entity type, subject, grounded evidence snippet, and a relevance score, ensuring accurate and explainable outputs tied to the source text.

Implementation

The pipeline implements a complete domain-specific knowledge base builder that processes raw XML court judgments, extracts legal facts using a prompt-driven schema, indexes them in a vector store, and enables precise semantic search with reranking. Below, each stage is broken down with detailed reasoning and code walkthroughs.

Step 1 — Environment setup and imports

```
!pip install -q langextract qdrant-client sentence-transformers beautifulsoup4
```

```
import os, re, uuid, textwrap
from lxml import etree
import langextract as lx
```

```
from langextract.inference import OpenAILanguageModel
from qdrant_client import QdrantClient
from qdrant_client.models import VectorParams, Distance,
PointStruct, Filter, FieldCondition, MatchValue
from sentence_transformers import SentenceTransformer
```

We prepare the environment by installing all required libraries: LangExtract for entity extraction, Qdrant for vector storage, Sentence-Transformers for embeddings, `lxml` package for XML parsing, and OpenAI/Google SDKs for LLM interaction. These imports bring in the core building blocks for the entire pipeline.

Step 2 — Prompt and few-shot examples

```
prompt_legal = textwrap.dedent("""\
Extract structured legal facts from the case text. Classes:
1) CaseName {name}
...
Rules:
- Use exact text spans from the original.
- Omit fields if not explicitly stated.
""")
```

This is the schema definition for the extraction. It defines the fact categories (e.g., CaseName, Date, Party, Legislation) and the rules for extraction.

```
examples_legal = [
    lx.data.ExampleData(
        text="Federal Court of Australia\nSmith v Jones [2020] FCA 123...",
        extractions=[ ... ]
    )
]
```

These few-shot examples guide the LLM to produce structured and traceable

Open in app ↗

Medium

🔍 Search



```
def get_text_from_xml(filepath):
    parser = etree.XMLParser(recover=True)
    tree = etree.parse(filepath, parser)
    root = tree.getroot()
    ...
    return "\n".join(line for line in all_text if len(line) > 50)
```

The `get_text_from_xml()` function is designed to parse an XML legal document, walk through every element, and return a single cleaned text string suitable for NLP processing. It begins by loading the file with `lxml.etree.XMLParser(recover=True)`, which ensures that even slightly malformed XML is handled gracefully—a common occurrence in large-scale scraped datasets.

Inside, a recursive helper function `recursive_text()` collects both the main text (`elem.text`) and any trailing text (`elem.tail`) from each XML node, cleaning out unwanted inline artifacts like `"id=c12">` using a regex, and stripping whitespace. This recursion allows the function to traverse deeply nested XML structures typical of legal cases.

Once all text segments are gathered, it filters out very short lines (less than 50 characters) to remove noise, then joins the remaining lines into a single cohesive string. The result is a clean, context-rich text representation of the legal document, ready for entity extraction or semantic search indexing.

Step 4 — Qdrant Collection setup and Embedding model

```
COLLECTION_NAME = "legal_facts"
qdrant = QdrantClient(path="/content/drive/MyDrive/qdrant_data")
embedder = SentenceTransformer("intfloat/e5-base-v2")
dim = embedder.get_sentence_embedding_dimension()
qdrant.recreate_collection(
    collection_name=COLLECTION_NAME,
    vectors_config=VectorParams(size=dim, distance=Distance.COSINE),
)
```

This snippet sets up the Qdrant vector database to store and search extracted legal facts. First, it defines the collection name `"legal_facts"` and initializes a Qdrant

client in local storage mode at the given path. Then, it loads the "intfloat/e5-base-v2" SentenceTransformer model, which will embed text into dense numerical vectors, and retrieves the embedding dimension (`dim`).

Finally, it recreates a Qdrant collection with that dimension and specifies **cosine similarity** as the distance metric, ensuring semantic searches are based on vector similarity rather than exact keyword matching.

Step 5 — Entity extraction and indexing

```
result = lx.extract(  
    text_or_documents=text,  
    prompt_description=prompt_legal,  
    examples=examples_legal,  
    language_model_type=OpenAILanguageModel,  
    model_id="gpt-4o-mini",  
    api_key=os.environ["OPENAI_API_KEY"],  
    fence_output=True,  
    use_schema_constraints=True,  
)
```

This code invokes **Google's LangExtract** to extract structured legal entities from the input `text`. It supplies the model with a carefully crafted prompt (`prompt_legal`) describing the entity types, their attributes, and formatting rules, along with `examples_legal` that serve as few-shot examples to guide consistent extraction. The extraction is powered by OpenAI's `gpt-4o-mini` model, specified through `language_model_type=OpenAILanguageModel`, and authenticated using the API key stored in the environment.

Parameters like `fence_output=True` ensure the results are returned in a structured, parseable format, while `use_schema_constraints=True` enforces strict adherence to the predefined schema. This setup guarantees that the extracted entities are accurate, consistent, and traceable back to the original text.

```
vectors = embedder.encode(evidence_texts, batch_size=32).tolist()  
points = []  
for vec, meta in zip(vectors, metas):  
    fact_id = str(uuid.uuid4())
```

```

payload = dict(meta)
payload["fact_id"] = fact_id
points.append(PointStruct(id=fact_id, vector=vec, payload=payload))
qdrant.upsert(collection_name=COLLECTION_NAME, points=points)

```

The above snippet is responsible for **embedding and storing extracted legal facts into Qdrant**. First, the list of evidence texts (`evidence_texts`) is transformed into high-dimensional numerical vectors using the `embedder.encode()` method, allowing semantic similarity search later. Each vector is paired with its corresponding metadata (`metas`), which contains details like entity type, attributes, and original character offsets.

For each fact, a unique identifier (`fact_id`) is generated to ensure distinct indexing. A `PointStruct` object is then created, combining the vector and its payload metadata. Finally, all these points are inserted (or updated, if already existing) into the Qdrant collection using the `upsert()` method, making the knowledge base ready for fast semantic retrieval.

Step 6 — Semantic search with re-ranking

```

nltk.download("punkt")

def get_full_context(doc_id, start_char, end_char, window=100):
    if doc_id not in source_docs:
        return None
    text = source_docs[doc_id]
    start = max(0, start_char - window)
    end = min(len(text), end_char + window)
    snippet = text[start:end].strip()
    sentences = nltk.sent_tokenize(snippet)
    return " ".join(sentences)

def rerank_results(query, results):
    rerank_input = [(query, r["evidence_text"]) for r in results]
    scores = reranker.rerank(query, [r["evidence_text"] for r in results])
    ranked = sorted(
        zip(results, scores),
        key=lambda x: x[1],
        reverse=True
    )
    return [
        {**res, "rerank_score": score}

```

```

        for res, score in ranked
    ]

def search_semantic(query, entity_type=None, limit=10):
    qvec = embedder.encode(query).tolist()
    qfilter = None
    if entity_type:
        qfilter = Filter(must=[FieldCondition(key="entity_type", match=MatchVal

    hits = qdrant.query_points(
        collection_name=COLLECTION_NAME,
        query=qvec,
        query_filter=qfilter,
        limit=limit,
        with_payload=True
    )

    results = []
    for p in hits.points:
        subj = p.payload["subject"]
        evidence = p.payload["evidence_text"]
        ctx = get_full_context(
            p.payload["doc_id"],
            p.payload["start_char"],
            p.payload["end_char"],
            window=200
        )
        if ctx:
            evidence = ctx

        results.append({
            "entity_type": p.payload["entity_type"],
            "subject": subj,
            "evidence_text": evidence,
            "score": float(p.score)
        })

    # === Rerank ===
    new_scores = list(reranker.rerank(query, [r["evidence_text"] for r in results]))
    for r, s in zip(results, new_scores):
        r["rerank_score"] = s

    results.sort(key=lambda x: x["rerank_score"], reverse=True)
    return results

```

This section of the code enhances the semantic search pipeline by ensuring that the retrieved results are both contextually complete and properly ranked for relevance. The `get_full_context` function plays a crucial role here by expanding the retrieved

text around the matched span. Instead of returning a clipped or fragmented phrase, it extracts a broader window of text from the original document and then tokenizes it into clean sentences. This ensures that the evidence presented to the user is intelligible, grammatically complete, and preserves the surrounding legal meaning rather than just an isolated fragment.

Once retrieval is done, the `rerank_results` function takes over. While the initial vector search through Qdrant identifies candidate matches based on embeddings, reranking allows a secondary layer of precision. This function pairs the query with each evidence snippet, then passes them through a dedicated reranker model, which assigns more nuanced similarity scores. These scores capture finer semantic relationships than the embedding similarity alone, such as whether the evidence truly addresses the intent of the query. The results are then sorted again, this time based on reranker confidence, ensuring that the most directly relevant legal text rises to the top.

The `search_semantic` function orchestrates the full pipeline by embedding the user's query, applying an optional filter on entity types, and querying Qdrant for potential matches. For each hit, it pulls in the broader context using `get_full_context`, creating evidence passages that are easier for legal practitioners to interpret. The final results are then passed through the reranking stage to refine ordering, producing a list of context-rich, relevance-sorted outputs. Together, these functions elevate the system from a simple vector retriever to a robust legal search engine that balances breadth of context with depth of relevance.

Step 7 — Generate answer by passing retrieved results to LLM

```
def generate_clean_answer(query, results):  
    context = format_results_for_llm(results, top_k=5)  
    prompt = f"""  
    You are a legal research assistant writing a professional case law summary.  
    A user has asked: "{query}".  
    Based on the retrieved legal evidence, write a clear, professional answer in plaintext.  
    - Focus only on the relevant facts and legal principles.  
    - Do not mention the retrieval process, evidence chunks, or that you are summarizing.  
    - Do not say things like "your query does not directly match".  
    - Write as if you are drafting a concise legal brief, suitable for a lawyer or judge.  
    - Keep the answer factual and objective.  
    - If the evidence is indirect, frame it as "Relevant cases indicate..." instead of "The evidence suggests...".  
    Evidence:
```

```

{context}
"""
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response.choices[0].message.content

```

This function transforms the retrieved search results into a polished legal answer. It first prepares the **context** by selecting and formatting the top five results, so the LLM can reason over them coherently. Then, it constructs a **prompt** that tells the model to act like a professional legal research assistant, with strict instructions not to mention evidence chunks, retrieval processes, or mismatches. This keeps the tone focused on facts and legal principles.

The call to `client.chat.completions.create` uses `gpt-4o-mini` with `temperature=0`, ensuring a precise and objective response. Finally, the model's answer is extracted and returned as clean text—ready to be presented as a concise legal summary.

Step 8 — Example search

```

results = search_semantic("application under section 79 of the Family Law Act")
answer = generate_clean_answer("application under section 79 of the Family Law Act")
print(answer)

```

An application under Section 79 of the Family Law Act pertains to the division

Key legal principles include:

1. Jurisdiction: The Family Court has the authority to determine property disputes.
2. Considerations for Orders: The court considers various factors when making property orders.
3. Equitable Distribution: The aim is to achieve a fair distribution of assets, including property.
4. Relevant Case Law: Relevant cases indicate that the court has discretion in making property orders.

In summary, an application under Section 79 of the Family Law Act is a formal request for the court to make property orders.

This output demonstrates how the system transforms retrieved legal evidence into a professional, concise summary suitable for lawyers, students, or researchers. Instead of raw text fragments, the LLM organizes the information into a structured legal brief that clearly explains the jurisdiction, guiding principles, and case law relevance under Section 79.

By combining semantic search, reranking, and LLM-based summarization, the pipeline ensures that complex case law is distilled into plain English, while preserving the nuance of legal reasoning. This makes the answer both authoritative and easy to understand, showing the practical value of AI in legal research.

For the full executable code with dataset, refer to my GitHub ->

<https://github.com/vlds18/Legal-search-engine-Qdrant-LangExtract>



Follow

Written by Vlds

5 followers · 3 following

No responses yet



José Manuel Sánchez

What are your thoughts?

More from Vlds



Vlds

MedQuery: Graph-Augmented Retrieval for Accurate Question Answering

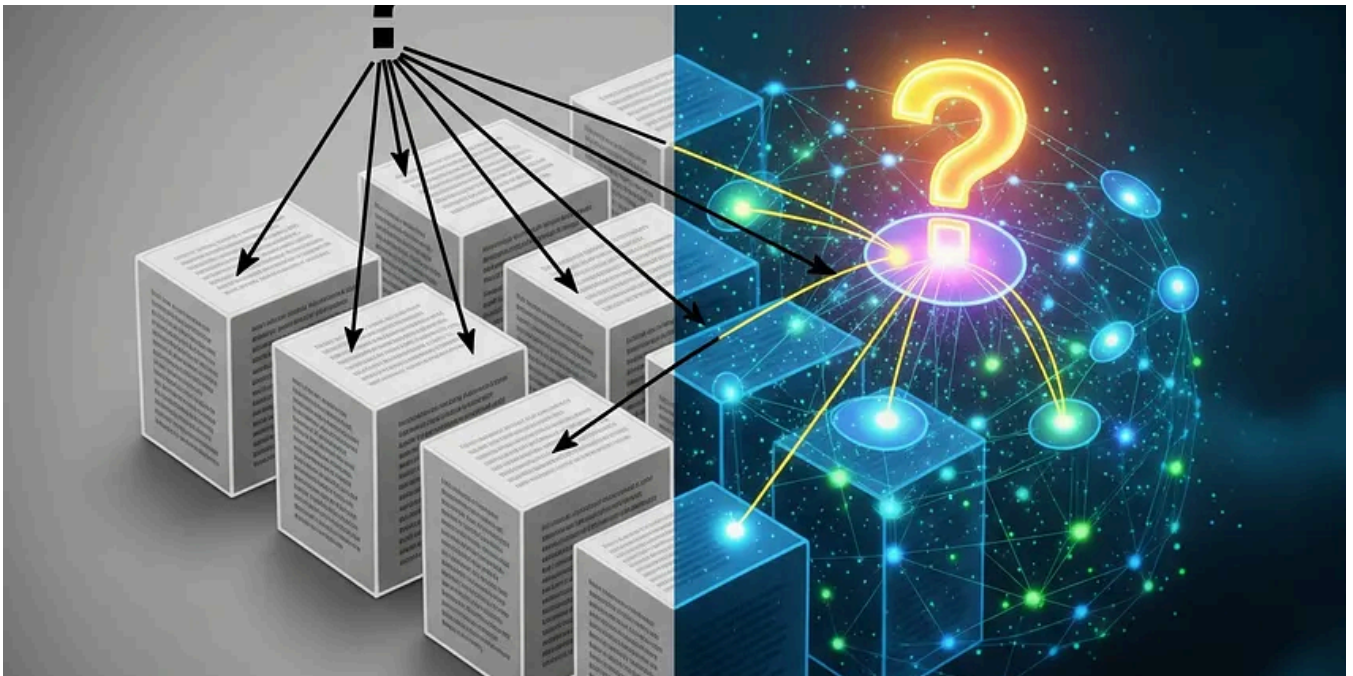
When answering clinical questions, precision isn't optional—it's mission-critical. MedQuery fuses knowledge graphs and vector search to...

Jul 29



See all from Vlds

Recommended from Medium



 In Neo4j Developer Blog by Tomaz Bratanic

From Legal Documents to Knowledge Graphs

Use LlamaCloud to extract information and construct legal graphs in Neo4j

5d ago  119

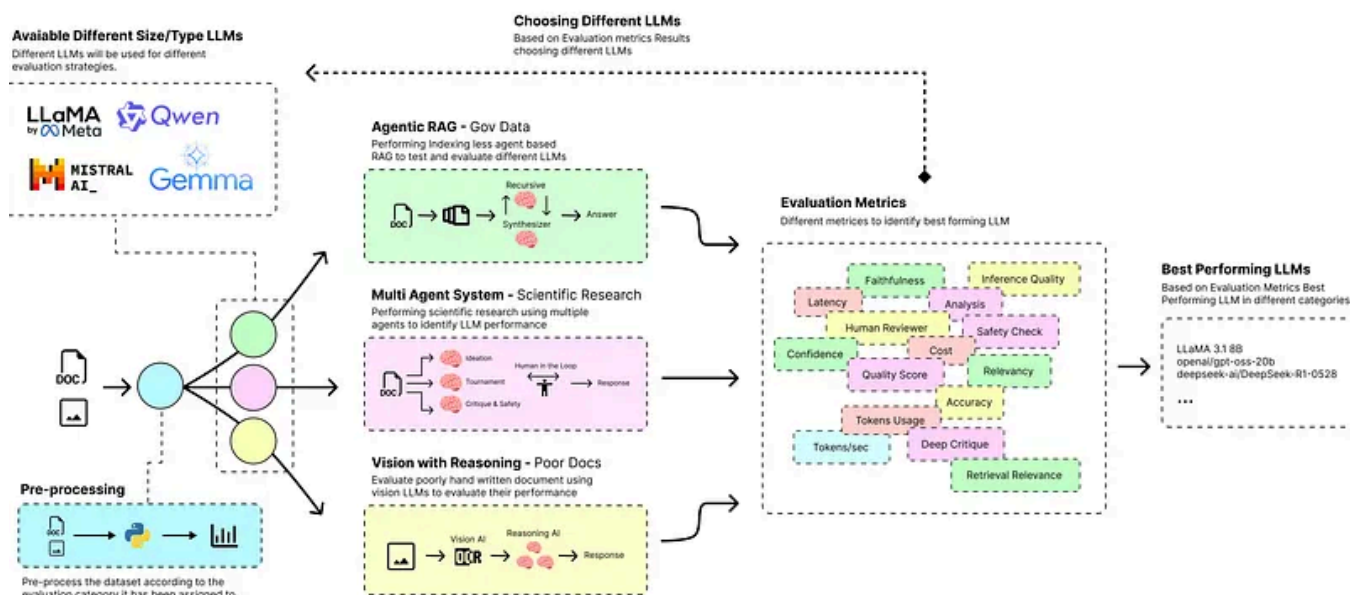


 In Codrft by Codrft

I Built an AI That Eats 1,000s of PDFs for Breakfast and Explains Them Better Than I Can

When I looked at my desktop last month, it felt like an archeological dig site of my research habits. Dozens (actually hundreds) of PDFs...

★ Aug 16 🖱 204 💬 8



In Level Up Coding by Fareed Khan

Building Three Pipelines to Select the Right LLMs for RAG, Multi-Agent Systems, and Vision

Agentic RAG, Multi-Agent and Vision with Reasoning

★ 4d ago 🖱 579 💬 8



In Towards Deep Learning by Sumit Pandey

Gemma 3 (270M) vs 1B vs 4B: The Tiny-Titan Showdown

This morning I saw linkedIn posts of people saying about how cool Gemma 3 (270M) is. While scrolling through the comments, one caught my...

★ Aug 18 🖱️ 22 💬 1



In Data Science in Your Pocket by Mehul Gupta

Qwen-Image-Edit : Best AI model for image editing

How to use Qwen-Image-Edit for free?

5d ago 🖱️ 71



Phase	CPU (ms)	T4 GPU (ms)
Embedding	320	72
Vector Search	110	110
Reranking	2600	420
LLM Gen	1800	1800



Marcus Feldman

Building a Production RAG System: Qwen3 Embeddings, Reranking, and Vector Database Insights

SECTION 1: PROJECT KICKOFF AND OBSERVATIONS

Aug 4  8



See more recommendations